

NIRBHAY

5-Month Lab-wise Development Roadmap

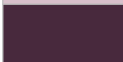
Schedule: 3 labs/week | 2 hours/lab | 20 weeks | 60 lab sessions total
Prepared for: Dhvani | Final Year Project (4CP31) | Version 1.0

How to Use This Roadmap

This plan breaks your 5-month semester into 20 weeks, with 3 labs a week (2 hours each) = 60 total sessions. It assumes you already know basic Flutter/Dart and have your UI screens designed in Google Stitch, so Phase 1 skips tutorial-style basics and goes straight into project setup and wiring real logic behind your existing Stitch screens. Each lab has a clear **Focus**, a short list of **Tasks** to complete in that 2-hour block, and a ready-to-use **AI Prompt** you can paste into Claude (or any AI tool) to generate the code. Don't just copy-paste the output - read through it, run it, and ask a follow-up question if any part doesn't make sense.

Tip: Keep a single running Flutter project from Week 1 onward, and commit your code to GitHub at the end of every lab, even if it's not perfect. It builds a clean history you can show your panel and gives you a safety net if something breaks later.

App Color Palette

Swatch	Name	Hex	RGB
	Thistle	#D8C0CB	216, 192, 203
	Old Burgundy	#48293D	72, 41, 61
	Light Blue	#AADBE3	170, 219, 227
	UCLA Blue	#506F8A	80, 111, 138
	Jakarta	#45304D	69, 48, 77

This palette is used as the reference whenever a lab in this plan mentions applying or matching the app's theme.

Tech Stack

Layer	Technology	Purpose
Frontend	Flutter	Cross-platform mobile UI for Android & iOS from a single codebase
Backend	Node.js + Express	REST APIs for alerts, contacts, timers and service coordination
Database	MySQL	Stores user profiles, trusted contacts, alert logs and reported locations
Real-time Alerts	Firebase Cloud Messaging / Socket.io	Pushes instant SOS notifications and streams live location to trusted c
Location & Maps	Google Maps / Geolocation APIs	Live GPS tracking and safety heatmap rendering
Authentication	JWT	Secure login and session handling

Phase Overview

Phase	Weeks	Focus
1. Project Setup & Stitch UI Integration	1-3	Flutter + Node/Express + MySQL setup, JWT auth, navigation
2. Trusted Contacts & Location	4-5	Contacts CRUD via API, geolocation, maps basics
3. SOS Alert & Live Location	6-8	Core SOS trigger, FCM alerts, Socket.io live location sharing
4. Voice/Shake Trigger & Safety Timer	9-11	Hands-free triggers, check-in timer, auto-escalation
5. Fake Call Generator	12	Simulated incoming call UI and logic
6. Nearby Emergency Services	13-14	Places API, helpline directory, map markers
7. Community Safety Module	15-16	Unsafe location reporting, safety tips feed
8. Integration & Testing	17-18	State management refactor, polish, bug fixing, APK build
9. Documentation & Presentation	19-20	Report writing, demo script, panel prep

PHASE 1 - Project Setup & Stitch UI Integration

Weeks 1-3

Week 1

Lab 1: Set up the Flutter project & import Stitch UI

- Install Flutter SDK and set up your emulator/device (skip this if already done).
- Create the nirbhay_app project and set up a clean folder structure (screens, widgets, models, services).
- Import your exported Stitch screens/assets and get them compiling inside the project.

AI PROMPT

I have UI screens exported from Google Stitch for my Flutter app called Nirbhay. Help me set up a new Flutter project with a clean folder structure (screens, widgets, models, services) and correctly import the exported Stitch code and assets into it.

Lab 2: Turn your first Stitch screen into a working Flutter screen

- Pick your splash or login screen from Stitch.
- Clean up the exported code into a proper Flutter widget file.
- Confirm it renders correctly on your emulator/device.

AI PROMPT

Here is the code Stitch generated for my [screen name] screen: [paste code]. Help me clean it up into a proper, well-structured Flutter widget and explain any parts of the generated code I should understand or change.

Lab 3: Wire up navigation across all Stitch screens

- Set up a bottom navigation bar or route structure for Home, Map, Community, Profile.
- Plug in your existing Stitch screens as the content for each route/tab.
- Test navigating between all screens.

AI PROMPT

Give me Flutter code for a bottom navigation bar / route setup with tabs for Home, Map, Community, and Profile, and show me how to plug in my existing Stitch-generated screens (code: [paste]) as the content for each tab, without changing their design.

Week 2

Lab 1: Set up the Node.js + Express backend & MySQL

- Create the backend project with a clean folder structure (routes, controllers, models, middleware).
- Connect it to a MySQL database using mysql2 or Sequelize.
- Add a /health endpoint and confirm the connection with a simple test query.

AI PROMPT

Help me set up a Node.js + Express backend for my Flutter app Nirbhay, with a clean folder structure (routes, controllers, models, middleware), and connect it to a MySQL database using mysql2 (or Sequelize). Include .env config and a /health test endpoint.

Lab 2: Design the users table & build JWT authentication

- Design a MySQL 'users' table (name, email, password hash, phone).
- Build signup and login endpoints with bcrypt password hashing.
- Generate and return a JWT on successful login.

AI PROMPT

Help me design a MySQL 'users' table and implement JWT-based signup and login endpoints in Node.js + Express, including password hashing with bcrypt and JWT token generation.

Lab 3: Wire your Stitch login/signup screens to the auth API

- Connect your existing Stitch Sign Up and Login screens to the new auth endpoints.
- Store the JWT securely using flutter_secure_storage.
- Save additional profile fields (name, phone) via a profile API call.

AI PROMPT

I already have a Stitch-designed Sign Up screen and Login screen in Flutter (code: [paste both]). Help me connect them to my Node/Express JWT auth API, storing the token securely using flutter_secure_storage, and saving additional profile fields (name, phone) via a profile API call, without changing the existing design.

Week 3

Lab 1: Apply the Nirbhay color palette app-wide

- Create a theme.dart file mapping your exact palette to Flutter's ThemeData.
- Define primary/secondary/background/accent roles for each color.
- Apply the theme consistently across your existing Stitch screens.

AI PROMPT

Help me build a Flutter theme.dart file using this exact color palette: Thistle (#D8C0CB), Old Burgundy (#48293D), Light Blue (#AADBE3), UCLA Blue (#506F8A), and Jakarta (#45304D). Map these to primary, secondary, background, surface, and accent roles in ThemeData, and show me how to apply this theme consistently across my existing Stitch-generated screens.

Lab 2: Wire logic behind the Home screen

- Take your Stitch-designed Home screen with the SOS button and quick-action icons.
- Connect each quick-action icon to real navigation (Fake Call, Timer, Map, Community).
- Leave the SOS button's actual trigger logic as a placeholder for Phase 3.

AI PROMPT

I have a Stitch-designed Home screen with an SOS button and quick-action icons (code: [paste]). Help me wire up real navigation and logic behind each element, keeping the SOS button as a placeholder for now, without changing the existing design.

Lab 3: Test full flow & push to GitHub

- Test signup -> login -> home navigation end-to-end.
- Fix any bugs found.
- Create a GitHub repo and push your project.

AI PROMPT

I'm getting [describe error] when testing my Flutter app's login-to-home navigation flow. Here's my code: [paste code]. Help me debug it. Also explain how to create a GitHub repository and push my Flutter project to it for the first time.

PHASE 2 - Trusted Contacts & Location

Weeks 4-5

Week 4

Lab 1: Trusted contacts table & API

- Design a MySQL 'trusted_contacts' table linked to users by foreign key.
- Build Express CRUD endpoints (add, list, delete), protected by JWT auth middleware.
- Test the endpoints with sample data.

AI PROMPT

Help me design a MySQL 'trusted_contacts' table (linked to users by foreign key) and build Express CRUD endpoints (add, list, delete) for managing a user's trusted contacts, protected by JWT auth middleware.

Lab 2: Import from phone contacts (optional)

- Add the flutter_contacts package.
- Let the user pick a contact from their phone's contact list.
- Save the picked contact via your trusted-contacts API.

AI PROMPT

Show me how to use the flutter_contacts package to let users pick a contact from their phone's contact list and save it as a trusted contact via my Express trusted-contacts API.

Lab 3: Trusted Contacts management screen

- Add edit and delete actions to the contacts list.
- Connect the screen fully to your Express API (with JWT auth header).
- Match styling to the app's existing color theme (Old Burgundy / UCLA Blue palette).

AI PROMPT

Help me build a Trusted Contacts screen in Flutter with edit and delete functionality, connected to my Express trusted-contacts API (sending the JWT in the auth header), matching my app's existing color theme (Thistle, Old Burgundy, Light Blue, UCLA Blue, Jakarta).

Week 5

Lab 1: Fetch current location

- Add the geolocator package.
- Request location permission properly.
- Fetch and print current latitude/longitude.

AI PROMPT

Give me Flutter code using the geolocator package to request location permission and fetch the user's current latitude and longitude. Explain permission handling for Android and iOS.

Lab 2: Display location on Google Map

- Get a Google Maps API key and set it up.
- Add google_maps_flutter to the project.
- Show the user's current location as a marker on the map.

AI PROMPT

Walk me through setting up a Google Maps API key and displaying the user's current location on a map using google_maps_flutter in my Flutter app.

Lab 3: Integration test & commit

- Test location fetch + map display together.
- Fix any permission or rendering bugs.
- Commit progress to GitHub.

AI PROMPT

My Google Map isn't showing my current location correctly, here's my code: [paste]. Help me fix it.

PHASE 3 - SOS Alert & Live Location

Weeks 6-8

Week 6

Lab 1: SOS trigger logic (client-side)

- Build the SOS button's tap handler.
- Fetch location and prepare an alert payload with a Maps link.
- Print/test the payload before wiring it to the backend next lab.

AI PROMPT

Give me Flutter code for an SOS button that, when tapped, fetches the current location and prepares an alert payload (user id, location, timestamp) with a Google Maps link, ready to send to a backend API.

Lab 2: Backend /sos/trigger endpoint + Firebase Cloud Messaging

- Build a POST /sos/trigger endpoint in Express.
- Look up the user's trusted contacts from MySQL.
- Send an FCM push notification with the location to each contact's device token.

AI PROMPT

Give me a Node.js + Express endpoint POST /sos/trigger that receives a user's SOS alert (location + user id), fetches their trusted contacts from MySQL, and sends a Firebase Cloud Messaging push notification with the location to each contact's registered device token.

Lab 3: Log alerts in MySQL & connect Flutter to the endpoint

- Design an 'sos_alerts' table (user, location, timestamp, status).
- Insert a new row whenever /sos/trigger is called.
- Call this endpoint from the Flutter SOS button and test with two accounts.

AI PROMPT

Help me design a MySQL 'sos_alerts' table to log each SOS event (user, location, timestamp, status), save a new row whenever POST /sos/trigger is called, and give me the Flutter code to call this endpoint when the SOS button is tapped, sending the JWT in the auth header.

Week 7

Lab 1: SMS fallback for low connectivity

- Add url_launcher package.
- Open the SMS app pre-filled with an emergency message + location link.
- Test this works without an active internet connection.

AI PROMPT

Show me how to use Flutter's url_launcher package to open the SMS app pre-filled with an emergency message and location link, so it works even without internet.

Lab 2: Live location sharing via Socket.io

- Add Socket.io to your Express backend.
- Broadcast the user's live location updates to their trusted contacts for a set duration.
- Emit location updates from the Flutter client to the socket.

AI PROMPT

Give me Node.js + Express + Socket.io code to broadcast a user's live location updates to their trusted contacts in real time for 10 minutes after SOS is triggered, and Flutter client code (using socket_io_client) to emit location updates to this socket.

Lab 3: Cancel alert & confirmation dialog

- Add a 5-second confirm/cancel window before SOS actually sends.
- Build the confirmation dialog UI.
- Call a /sos/cancel endpoint to stop the alert and location broadcast if cancelled.

AI PROMPT

Help me add a cancel/confirm dialog in Flutter that appears for 5 seconds after SOS is triggered, allowing the user to cancel it before the alert is sent. Also give me a POST /sos/cancel Express endpoint that marks the alert as cancelled in MySQL and stops the Socket.io location broadcast.

Week 8

Lab 1: Trusted contact's live-view screen

- Build a screen for a trusted contact to view live location on a map.
- Connect a Socket.io client to receive real-time location updates.
- Test with a second device/account.

AI PROMPT

Give me Flutter code for a screen where a trusted contact can view a live-updating location on Google Maps, received in real time via a Socket.io client (socket_io_client) connected to my Node/Express backend.

Lab 2: Polish SOS UI/UX

- Add animation and haptic feedback to the SOS button.
- Add a clear confirmation/status screen after triggering.
- Keep everything within the app's existing color theme.

AI PROMPT

Suggest simple UI/UX improvements (animation, haptic feedback, visual state changes) for an SOS button in Flutter to make it feel reassuring and clear during an emergency, keeping it consistent with my app's color theme (Old Burgundy accents on a Thistle/Light Blue base).

Lab 3: Full SOS flow testing

- Test trigger, alert delivery, live location, and cancel end-to-end.
- Test edge cases: permission denied, no internet.
- Log and prioritize any bugs found.

AI PROMPT

I want to test my full SOS flow end-to-end. Give me a checklist of test cases to verify (permissions denied, no internet, cancel mid-alert, etc.) for a Flutter emergency alert feature.

PHASE 4 - Voice/Shake Trigger & Safety Timer

Weeks 9-11

Week 9

Lab 1: Shake detection

- Add the sensors_plus package.
- Detect shake gestures using accelerometer data.
- Trigger a test function when shake threshold is crossed.

AI PROMPT

Give me Flutter code using the sensors_plus package to detect when the user shakes their phone and trigger a function (like SOS) when shake intensity crosses a threshold.

Lab 2: Background shake detection

- Understand Android foreground services basics.
- Make shake detection work while the app is backgrounded.
- Test on a real device if possible (emulators can be unreliable for sensors).

AI PROMPT

Explain how to make shake detection work in Flutter even when the app is in the background, using a foreground service on Android. Give me starter code.

Lab 3: Tune shake sensitivity

- Test shake triggers across different real-world scenarios.
- Adjust threshold values to reduce false positives/negatives.
- Document the final threshold and reasoning.

AI PROMPT

My shake detection triggers too easily / not easily enough in Flutter. Here's my threshold logic: [paste code]. How do I tune it better?

Week 10

Lab 1: Voice command detection

- Add the speech_to_text package.
- Listen continuously for a specific safe-word phrase.
- Call a test function when the phrase is detected.

AI PROMPT

Give me Flutter code using the speech_to_text package to continuously listen for a specific trigger phrase (like 'help me now') and call a function when it's detected.

Lab 2: Unified emergency trigger service

- Refactor shake + voice detection into one EmergencyTriggerService class.
- Make both call the same underlying SOS function.
- Clean up duplicate code.

AI PROMPT

Help me refactor my shake detection and voice detection code into a single reusable Flutter service class called EmergencyTriggerService that calls the same SOS function.

Lab 3: Handle permission edge cases

- Test what happens if mic/location permission is denied.
- Add helpful in-app messages guiding the user to enable permissions.
- Re-test both triggers together.

AI PROMPT

How do I gracefully handle microphone permission denial in Flutter's speech_to_text package and show a helpful message to the user?

Week 11

Lab 1: Safety Timer screen

- Build a screen to set a countdown duration.
- Show a live countdown on screen.
- Store the active timer state.

AI PROMPT

Give me Flutter code for a Safety Timer screen where a user sets a countdown duration (e.g. 30 minutes) and sees a live countdown timer on screen.

Lab 2: Check-in & auto-escalation

- Send a local notification 2 minutes before timer ends asking for confirmation.
- Auto-trigger SOS if the user doesn't respond in time.
- Test both the confirm and no-response paths.

AI PROMPT

Give me Flutter code that sends a local notification asking the user to confirm they're safe 2 minutes before their safety timer ends, and automatically triggers the SOS alert function if they don't respond in time.

Lab 3: Extend/confirm buttons & polish

- Add 'I'm Safe' and 'Extend Timer' buttons.
- Connect both to existing timer logic.
- Polish the timer screen UI.

AI PROMPT

Help me add 'Extend Timer' and 'I'm Safe' buttons to my safety timer notification/screen in Flutter, connected to my existing timer logic.

PHASE 5 - Fake Call Generator

Week 12

Week 12

Lab 1: Fake call screen UI

- Build a full-screen UI mimicking a native incoming call.
- Add caller name, photo, and Accept/Decline buttons.
- Test the visual on different screen sizes.

AI PROMPT

Give me Flutter code for a fake incoming call screen UI that looks like a real phone call screen, with caller name, photo, and Accept/Decline buttons.

Lab 2: Scheduled trigger, ringtone & vibration

- Add a delay picker to schedule the fake call.
- Play a ringtone using audioplayers.
- Add vibration using the vibration package.

AI PROMPT

Show me how to schedule a fake call to trigger after a delay in Flutter (e.g. after 30 seconds), and play a ringtone sound with vibration using the audioplayers and vibration packages.

Lab 3: Customizable caller settings

- Build a settings screen for caller name/photo.
- Save preferences locally with shared_preferences.
- Test end-to-end: schedule -> ring -> answer/decline.

AI PROMPT

Give me Flutter code for a settings screen where the user can customize the fake caller's name and photo, saved locally using shared_preferences.

PHASE 6 - Nearby Emergency Services

Weeks 13-14

Week 13

Lab 1: Google Places API integration

- Enable Places API in Google Cloud console.
- Fetch nearby police stations within a 5km radius.
- Parse and print name, address, distance.

AI PROMPT

Give me Flutter code to call the Google Places API and fetch nearby police stations within a 5km radius of the user's current location, returning name, address, and distance.

Lab 2: List + map markers

- Display results as a scrollable list.
- Also show them as markers on the map screen.
- Sync selection between list and map.

AI PROMPT

Help me display Google Places API results as both a scrollable list and markers on a Google Map in Flutter, syncing selection between the two.

Lab 3: Call & directions buttons

- Add a 'Call' button using `url_launcher` to open the dialer.
- Add a 'Get Directions' button opening the Maps app.
- Test both actions on a real device.

AI PROMPT

Give me Flutter code to add 'Call' and 'Get Directions' buttons to each emergency service list item, using `url_launcher` to open the dialer and Google Maps app.

Week 14

Lab 1: Helpline directory

- Build a MySQL 'helplines' table and a GET /helplines endpoint.
- Add a search/filter text field in the app.
- Add one-tap calling for each entry.

AI PROMPT

Give me a MySQL 'helplines' table design and an Express GET /helplines endpoint, plus Flutter code for a searchable list screen of emergency helpline numbers, with a text field to filter by name, and one-tap calling for each entry.

Lab 2: Loading, empty & error states

- Add a loading indicator while fetching places.
- Add an empty state for 'no results found'.
- Add error handling for no internet connection.

AI PROMPT

Help me add proper loading states, empty states, and error handling (no internet, no results found) to my nearby emergency services screen in Flutter.

Lab 3: Full testing & bug fixes

- Test maps, places, and helplines together.
- Fix any bugs found.
- Commit progress to GitHub.

AI PROMPT

I'm facing [describe the bug] with my Google Places integration in Flutter, here's my code: [paste]. Help me debug it.

PHASE 7 - Community Safety Module

Weeks 15-16

Week 15

Lab 1: MySQL schema for community reports

- Design fields: location (lat/lng), description, timestamp, reporter ID, status.
- Create the MySQL table with appropriate indexes for location queries.
- Write a short data-model note for your report.

AI PROMPT

Help me design a MySQL schema for a community safety reporting feature - a table for unsafe location reports (location, description, timestamp, reporter, status) with appropriate foreign keys and indexes for location-based queries.

Lab 2: Report unsafe location screen

- Let the user tap on the map to drop a pin.
- Add a text field for a short description.
- Save the report via a POST /reports endpoint.

AI PROMPT

Give me Flutter code for a screen where a user can tap on a Google Map to drop a pin at an unsafe location and add a short text description, then give me the Express + MySQL endpoint to save this report.

Lab 3: Show reports on the map

- Fetch all community reports via a GET /reports endpoint.
- Display them as markers, refreshing periodically or via Socket.io.
- Test that new reports appear promptly.

AI PROMPT

Show me how to fetch community safety reports from a MySQL-backed Express API and display them as markers on a Google Map in Flutter, refreshing periodically or via Socket.io for near real-time updates.

Week 16

Lab 1: Safety tips feed

- Create a MySQL 'tips' table.
- Build a GET /tips endpoint.
- Build a scrollable feed screen showing title + short text.

AI PROMPT

Give me a MySQL 'tips' table design and an Express GET /tips endpoint, plus Flutter code for a scrollable feed screen showing safety tips fetched from it.

Lab 2: Moderation & validation

- Add a 'report this post' button that flags a report row for review.
- Add basic validation to prevent empty/spam reports.
- Test the flagging flow.

AI PROMPT

Help me add a 'report this post' feature - an Express PATCH endpoint that flags a MySQL report row for admin review, called from a button in my Flutter community reports screen, plus basic input validation to prevent empty/spam reports.

Lab 3: Full module testing

- Test reporting, viewing, and flagging together.
- Fix bugs found during testing.
- Commit progress to GitHub.

AI PROMPT

Give me a checklist to test my community safety reporting feature for bugs and edge cases before moving to final integration.

PHASE 8 - Integration, Testing & Polish

Weeks 17-18

Week 17

Lab 1: State management refactor

- Introduce Provider (or Riverpod) for shared app state.
- Migrate SOS, Timer, and Contacts features to use it.
- Test that nothing breaks after the refactor.

AI PROMPT

Help me refactor my Flutter app to use Provider (or Riverpod) for state management across my SOS, Timer, and Contacts features, since currently everything is managed with local setState.

Lab 2: App-wide UI polish

- Review theming, fonts, and spacing across all screens.
- Standardize icon usage.
- Fix any visual inconsistencies.

AI PROMPT

Review my Flutter app's theme file and suggest improvements for consistent styling across all screens using my palette (Thistle, Old Burgundy, Light Blue, UCLA Blue, Jakarta) - spacing, typography, and icon usage.

Lab 3: Performance & battery optimization

- Review background services for battery drain.
- Reduce unnecessary API calls and add basic caching to frequent MySQL queries.
- Re-test app responsiveness.

AI PROMPT

Give me tips and code changes to reduce battery usage from my shake-detection and voice-detection background services in Flutter, and to add basic caching in my Express backend for frequently-hit MySQL queries.

Week 18

Lab 1: Full functional testing

- Test every feature end-to-end.
- Note all bugs found with steps to reproduce.
- Prioritize bugs by severity.

AI PROMPT

Give me a comprehensive test case checklist covering all features of my Flutter safety app (SOS, timer, fake call, maps, community) to test before final submission.

Lab 2: Bug fixing sprint

- Work through the prioritized bug list from Lab 1.
- Re-test each fix.
- Update your bug log.

AI PROMPT

Here's a bug I found in [feature]: [describe]. Here's the relevant code: [paste]. Help me fix it.

Lab 3: Deploy backend, branding & release build

- Deploy your Node.js + Express backend and MySQL database (e.g. Render/Railway).
- Add a custom app icon and splash screen branding.
- Generate a release APK and test it against the deployed backend.

AI PROMPT

Walk me through deploying my Node.js + Express backend and MySQL database to a free-tier host like Render or Railway, including environment variable setup. Then walk me through adding a custom app icon and splash screen to my Flutter app, and generating a release APK pointed at the deployed backend URL.

PHASE 9 - Documentation & Presentation

Weeks 19-20

Week 19

Lab 1: Technical documentation

- Write an architecture overview: Flutter + Node/Express + MySQL + FCM/Socket.io + Maps.
- Describe each module briefly.
- Add a simple architecture diagram.

AI PROMPT

Help me write a clear technical architecture section for my final year project report, describing how Flutter, Node.js/Express, MySQL, Firebase Cloud Messaging, Socket.io, and Google Maps API work together in my app, based on this feature list: [paste features].

Lab 2: Demo script

- List the key flows to showcase live.
- Write a 5-7 minute walkthrough script.
- Rehearse once with a timer.

AI PROMPT

Help me write a 5-7 minute live demo script for my final year project presentation, showcasing the SOS alert, safety timer, fake call, and community reporting features in a logical order.

Lab 3: Backup demo video

- Record a clean run-through of the app.
- Keep it under 5 minutes.
- Save it in at least two places (drive + laptop).

AI PROMPT

Give me tips on how to record a clean, professional demo video of my Flutter app for backup purposes in case my live demo fails during presentation.

Week 20

Lab 1: Finalize project report

- Expand your PRD into a full report format.
- Add results, screenshots, and conclusion.
- Proofread for consistency.

AI PROMPT

Help me expand my existing PRD into a full final year project report format including introduction, literature review, methodology, results, and conclusion sections.

Lab 2: Presentation & Q&A; prep

- Finalize presentation slides.
- Prepare answers for likely panel questions.
- Do a full rehearsal with timing.

AI PROMPT

Give me a list of 15 likely panel questions for a final year project on a women's safety Flutter app, along with strong sample answers.

Lab 3: Final buffer & submission checklist

- Do a last pass on code, docs, and APK.
- Back up all submission files.
- Do one final full rehearsal of the demo.

AI PROMPT

Give me a final pre-submission checklist for a Flutter final year project - code, documentation, APK, presentation, and backups.

Note: This schedule assumes an average pace. If a lab runs long, spill leftover tasks into the next session rather than rushing - the buffer in Phase 9 can absorb small delays.

— End of Document —